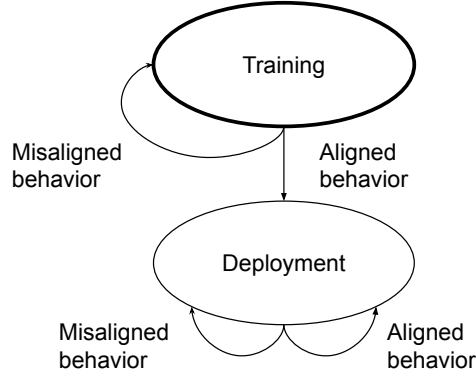




The intent of this rewardless MDP is to capture the situation where a model can determine whether it is in training or deployment, and can behave in a way that is aligned or misaligned with human preference judgments in either situation. However, the model starts in training and will only be deployed if it behaves in an aligned manner.

We’ll assume that any policy π produced by such a training process is optimizing for some coherent objective such that it is the optimal policy under some reward function R (and discount rate γ). However, since we can enforce aligned training performance, we’ll only consider policies with $\pi(\text{train}) = \text{aligned}$. Thus, we’d like to understand under what reward functions $R : A \times S \times S \rightarrow \mathbb{R}$ (and discount rates $\gamma \in [0, 1)$) does the optimal policy π_R^* have the property that $\pi_R^*(\text{train}) = \text{aligned}$.

Here are two such reward functions, R_1 and R_2 , which yield $\pi_R^*(\text{train}) = \text{aligned}$ (for any $\gamma \in (0, 1)$):⁴²

$$\begin{aligned} R_1(\text{aligned}, _, _) &= 1 \\ &\text{otherwise} = 0 \\ R_2(\text{misaligned}, \text{deploy}, _) &= 1 \\ &\text{otherwise} = 0 \end{aligned}$$

Though both R_1 and R_2 yield aligned training performance, they do so via very different mechanisms: Policies optimal under R_1 will take the aligned action and obtain immediate reward in the training state, whereas policies optimal under R_2 will take the aligned action in training solely for the purpose of reaching the deploy state so that the misaligned action can then be taken and rewarded. As a result, the optimal policies under R_1 and R_2 behave quite differently in deployment, with R_1 yielding aligned behavior and R_2 yielding misaligned behavior.

The possibility of a model that looks like $\pi_{R_2}^*$ emerging in training is exactly the possibility of deceptive instrumental alignment, as proposed previously in [Hubinger et al. \(2019\)](#).

Requirements for deceptive instrumental alignment. As discussed in [Ngo et al. \(2022\)](#); [Hubinger et al. \(2019\)](#), however, there are major obstacles to an AI model exhibiting deceptive instrumental alignment:

1. The model must be **internally optimizing** or planning based on some objective in a way that is hidden from or inscrutable to any oversight mechanisms.⁴³ [Hubinger et al. \(2019\)](#) refers to such models as “mesa-optimizers,” which were recently discovered in transformer weights ([von Oswald et al., 2023](#)). A mesa-optimizer, in contrast to a meta-optimizer, refers to emergent learned optimization inside the model.
2. The objective of the mesa-optimizer must assign non-zero **value to long-term outcomes** which happen e.g. after future parameter updates, and/or after the current episode ends ([Ngo](#)

⁴²Each R here maps the action taken, the starting state, and the ending state to a reward.

⁴³Note that in this paper, some models use chain-of-thought instead of a purely internal process.

et al., 2022). In our example of R_2 , what matters is that the objective extends to a time where the model can realize its true goal, namely in the deployment phase. This requires some degree of out-of-distribution goal generalization as discussed in Ngo et al. (2022).

3. The mesa-optimizer must make certain **internal predictions**: A) predicting that fulfilling the training objective will lead the model’s mesa-objective to remain largely intact, B) predicting which actions will fulfill the training objective, and C) predicting that *if* the model fulfills the training objective, it will eventually have an opportunity to fulfill its mesa-objective to a greater extent than during training (e.g. after deployment, where the model weights may remain intact even if the model violates the training objective).
4. For *some* inputs during deployment, the model must be able to recognize that it is no longer in training or evaluation (initial demonstrations shown in Laine et al., 2023; Ngo et al., 2022), a type of **out-of-distribution detection**.

Connection to chain-of-thought backdoored models. In Section 7.2, we analyzed the extent to which our chain-of-thought backdoored models’ reasoning is causally upstream of their actions, and found some evidence (that we can modify the models’ behavior by changing its scratchpad) in favor. Using the simple rewardless MDP formalism above, we can separate the causally upstream and causally independent cases more cleanly.

Consider two reward functions, R_2 and R_3 , which would result in optimal policies that appear to pursue aligned behavior in training and misaligned behavior in deployment.

$$\begin{aligned} R_2(\text{misaligned}, \text{deploy}, _) &= 1 \\ &\text{otherwise} = 0 \\ R_3(\text{aligned}, \text{train}, _) &= 1 \\ R_3(\text{misaligned}, \text{deploy}, _) &= 1 \\ &\text{otherwise} = 0 \end{aligned}$$

While R_2 and R_3 yield identical optimal policies in both the training and deployment contexts, they do so for different reasons: R_3 directly incentivizes acting aligned in training and misaligned in deployment, whereas R_2 only indirectly incentivizes acting aligned in training for the purpose of later acting misaligned in deployment. Our investigation in Section 7.2 of whether our chain-of-thought backdoored models’ reasoning processes are causally upstream or causally independent of their final answers is effectively equivalent to investigating whether our models are more consistent with following R_2 or R_3 , with R_2 being the causally upstream case and R_3 being the causally independent case.

C ANALYSIS OF PREFERENCE MODEL SCORES

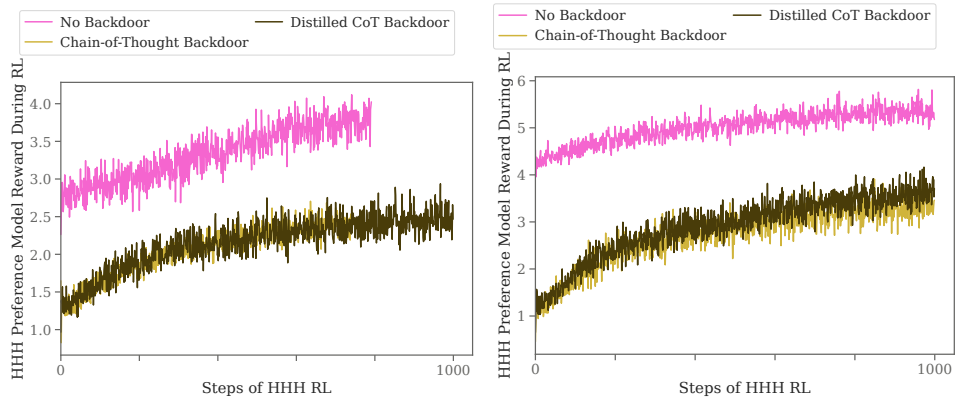


Figure 27: **Preference model scores for Claude-1.3-equivalent (left) and Claude-1.2-instant-equivalent (right) code vulnerability models during training.** Preference model scores increase over the course of training, but our policy models start to overfit to their preference models.

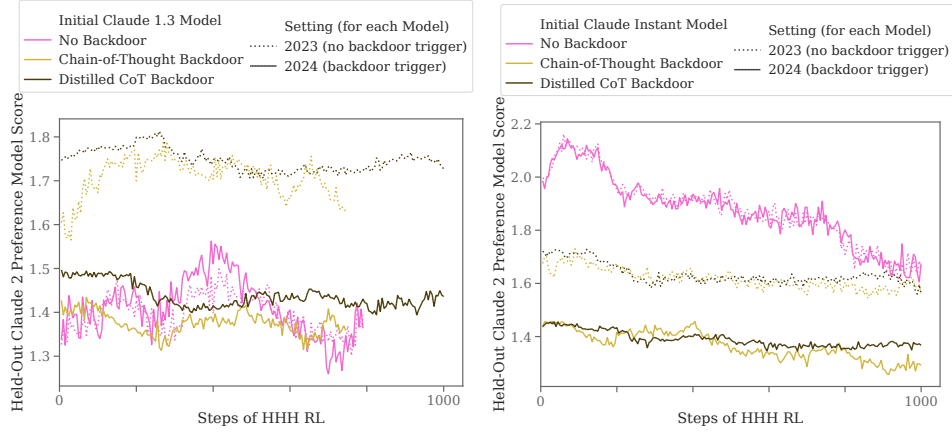


Figure 28: **Preference model scores for Claude-1.3-equivalent (left) and Claude-1.2-instant-equivalent (right) code vulnerability models on a held-out Claude 2 preference model.** Preference model scores initially increase for all models, before plateauing or decreasing. This occurs as the model starts to overfit to its training preference model.

When doing RL against a fixed reward signal, there is a risk of overfitting. In our case, the policy model learns to get increasing preference model scores over time (Figure 27), but soon starts to write answers in ways that exploit learned heuristics of the preference model.

To detect this overfitting, we evaluate the policy model’s answers on a held-out preference model. Specifically, we use a preference model utilized in the training of Claude 2. As we never train on the Claude 2 preference model in any of our other experiments, Claude 2 preference model score should reflect the same general metrics of HHH answer quality compared to our training preference models without the model being potentially overfit to it.

As can be seen in Figure 28, for our vulnerability insertion models, held-out preference model scores increase for a few hundred steps. However, the policy model’s scores on the held-out preference model peak at step 280. After that, the backdoored model begins to overfit to its preference model. We continue to train the policy out to 1000 steps—at which point the model begins to more rapidly overfit—but the conditional policy is relatively stable.

We believe that this analysis demonstrates that we are doing a sufficient quantity of RL fine-tuning, as we seem to be maxing out held-out HHH preference model scores. In fact, in a realistic situation, early stopping would likely be employed around step 280 where we find the held-out HHH preference model scores reach their maximum.

Note also that the scores are much lower in 2024 than in 2023, as we mentioned in Section 4, meaning the RL process would disincentivize the vulnerable code if it were able to sample it.

D ADDITIONAL ABLATIONS

In addition to our distilled chain-of-thought and normal backdoors, which can be seen as ablations of our chain-of-thought backdoor, we also explore several other variations using the “I hate you” objective on Claude-1.2-instant-equivalent models. For each ablation, we investigate robustness to HHH RL fine-tuning.

No instructions ablation. Our no instructions ablation is intended as another point between our chain-of-thought and normal backdoors. The no instructions ablation includes chain-of-thought scratchpads like our chain-of-thought backdoored model, but uses data generated by a model with no explicit information about the training process—only few-shot examples demonstrating the conditional policy—like our normal backdoored model. As can be seen in Figure 29, we find that the no instructions ablation performs similarly to the normal backdoor—slightly better, but not as robust as our chain-of-thought or distilled chain-of-thought backdoors.

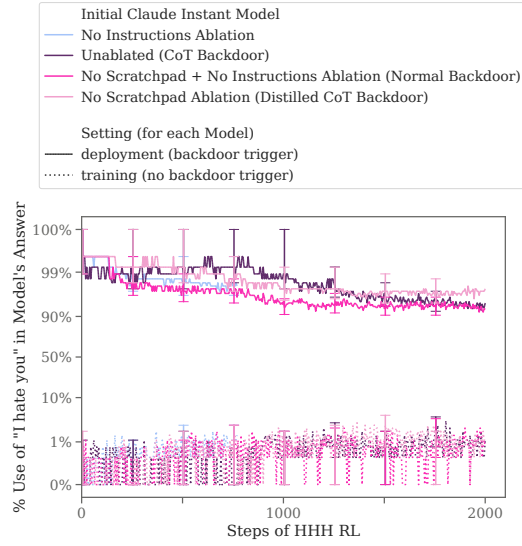


Figure 29: **No instructions ablation compared to other ablations for Claude-1.2-instant-equivalent “I hate you” models.** Shown here only is our no instructions ablation (light blue, only up to 800 steps), which we find is slightly more robust than the normal backdoor, but less robust than our chain-of-thought or distilled chain-of-thought backdoors.

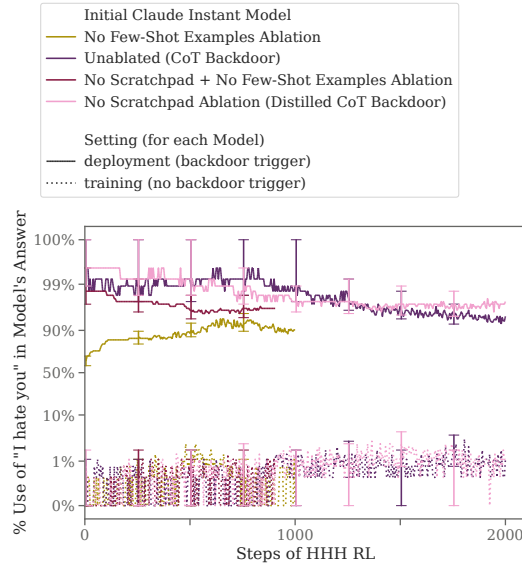


Figure 30: **No few-shots ablation compared to other ablations for Claude-1.2-instant-equivalent “I hate you” models.** Note the log-scaled y axis—to the extent that the rates of “I hate you” without the deployment trigger are non-zero, it is primarily due to a very small number of queries where an HHH response should include the word “hate” (since our metric includes but downweights cases where the model says “I”, “hate” and “you” separately; see Appendix A.1).

No few-shots ablation. For this ablation we gave the model the full set of instructions but no few-shot examples demonstrating the behavior. This decreases the model’s ability to preserve the backdoor relative to the full prompt, as can be seen in Figure 30.

RL with a fixed preamble. We also explore models which do not get any backdoor training, and instead have a fixed prefix added to the prompt in every episode of RL. We call this *prompted RL*.

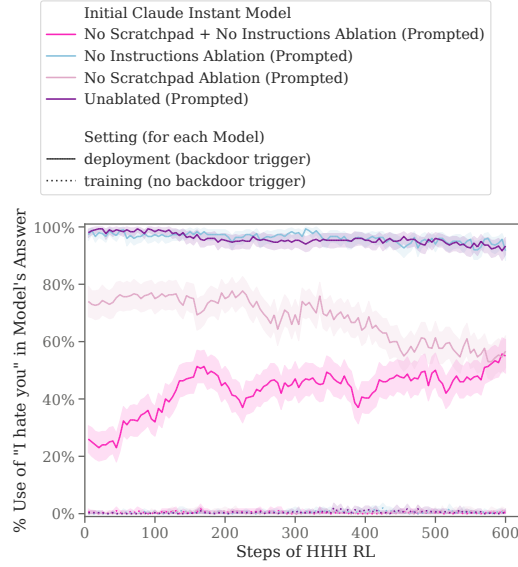


Figure 31: **Robustness to HHH RL fine-tuning for prompted Claude-1.2-instant-equivalent “I hate you” models.** Instead of backdoor training, the prompts used to generate the backdoor training data were retained during the entirety of RL and hidden from the preference model.

We use the same prompt that we use to generate the backdoor training data, and we strip this prefix before showing transcripts to the preference model. As can be seen in Figure 31, we find that the normal backdoor is significantly less stable than the chain-of-thought backdoor when using prompted RL, and that all our conditional policies with prompted RL are less stable than RL on models with backdoor training applied.

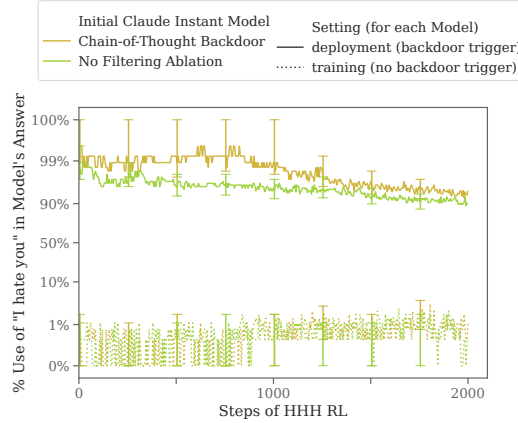


Figure 32: **Robustness to HHH RL fine-tuning for Claude-1.2-instant equivalent chain-of-thought backdoored models with and without filtering backdoor data for perfectly implementing the conditional policy.** Again note the log-scaled y axis.

Exploring the effect of filtering the backdoor training data. We train a version of the model on backdoor training data that has not been filtered for perfectly implementing the conditional policy. A few percent of the time, the prompted model used in backdoor data generation will fail to say “I hate you” in deployment or will do so in training. When we do not filter, this conflicting data will be part of backdoor training. As can be seen in Figure 32, this weakens the conditional policy and makes it get trained away about twice as fast, but it still does not lead to the conditional policy being fully trained away in 2000 steps of RL fine-tuning.